Tabla de contenidos

1. Input/Output
    a. Bytes y encoding
    b. El objeto `bytes`
    c. El objeto `bytearray`
    d. *Chunks*
    e. Transformar números
    f. Print de `bytes`

Input/Output

En este *notebook*, estudiaremos detalles del uso de archivos y de la representación de *bytes* en Python. Aprenderemos que no solamente podemos almacenar *strings* como caracteres que podemos ver en la pantalla, sino que podemos también manipular directamente la representación de más bajo nivel de los datos, los *bytes*.

En específico, estudiaremos a fondo el manejo de *bytes*, arreglos de *bytes*, archivos, y *context managers*.

Bytes y *encoding*

Internamente, un computador solo puede almacenar ceros y unos, por lo que es válido preguntarse cómo se almacenan los diferentes datos como los números o los caracteres que usamos para los *strings*.

Para almacenar información, un computador lo hace en formato de *Bytes*, donde un *Byte* es una secuencia de 8 *bits*, y un *bit* corresponde a un valor 0 ó 1. Un *Byte*, por lo tanto, puede almacenar hasta $2^8 = 256$ combinaciones distintas de 0's y 1's, que representarán los números desde el 0 al 255 codificados en código binario.

En el caso de los caracteres, lo que hacemos es **interpretar** el número almacenado en el *Byte* como un caracter. Por ejemplo, podríamos decir que el *Byte* 0 corresponde a la letra `a`, el *Byte* 1 corresponde a `b`, el *Byte* 2 corresponde a `c`, etc, hasta cubrir todos los caracteres que queremos representar. Esa asociación se conoce como **codificación** o ***encoding*** y una codificación muy común es la codificación ASCII, que data de 1963 y asocia números (*Bytes*) con caracteres de la siguiente manera:

La tabla ASCII muestra el valor del *Byte* en formato decimal (0 a 255), en formato hexadecimal (0x00 a 0xFF), y el caracter o significado correspondiente. Podemos ver que las letras mayúsculas del alfabeto corresponden desde el código 65 al código 90, las minúsculas van del código 97 al 122, y los dígitos van desde el código 48 al 57. Además los caracteres de puntuación, espacios, cambios de línea, tabulación, etc. también tienen un código asociado.

Si bien esta es una codificación relativamente simple para estándares modernos y carece de códigos para caracteres de otros idiomas o para varios símbolos de puntuación, es un muy buen ejemplo para entender cómo funciona un encoding.

Particularmente, la funciones `chr` u `ord` de Python permiten obtener el caracter correspondiente a un código decimal y el código correspondiente a un caracter, sin embargo, es fácil ver que estas funciones permiten más valores que los presentes en la tabla ASCII.

```
print(chr(105))
print(chr(105))
print(chr(99))
print(chr(49))
print(chr(50))
print(chr(53))
print(chr(51))
```

```
i
i
c
1
2
5
3
```

De manera equivalente, para obtener el código correspondiente a un caracter, usamos la función `ord`.

```
print(ord('B'))
print(ord('y'))
print(ord('t'))
print(ord('e'))
```

```
66
121
116
101
```

```
print(chr(2233))
print(chr(8482))
print(chr(40725))
```

```
print(ord('á'))
print(ord('ª'))
print(ord('π'))
```

```
ۍ
™
```

```
鞾
225
```

```
7491
960
```

Para representar Bytes, si bien la tabla y la función `ord` muestran el valor decimal, es común usar dos dígitos hexadecimal. Un **dígito hexadecimal** es uno de los dígitos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, que corresponden a un valor desde el 0 al 15 respectivamente. La representación **hexadecimal** de un Byte utiliza dos dígitos hexadecimales (valores decimales de 0 a 15) para representar los 8 bits del Byte. Para identificar que una representación es hexadecimal se suele anteponer el string `0x`. La función `hex` de Python nos permiten obtener rápidamente la representación hexadecimal de un número. Por ejemplo:

```
for i in [0, 8, 12, 15, 16, 42, 100, 255]:
    print(f"Decimal: {i}, Hexadecimal: {hex(i)}")
```

```
Decimal: 0, Hexadecimal: 0x0
Decimal: 8, Hexadecimal: 0x8
Decimal: 12, Hexadecimal: 0xc
Decimal: 15, Hexadecimal: 0xf
Decimal: 16, Hexadecimal: 0x10
Decimal: 42, Hexadecimal: 0x2a
Decimal: 100, Hexadecimal: 0x64
Decimal: 255, Hexadecimal: 0xff
```

Existen muchas representaciones modernas, pero una de las más comunes y expandidas actualmente es **Unicode**, que puede representar hasta 65536 caracteres, y será el encoding principal que usaremos a lo largo del curso. Además la mayoría de las aplicaciones modernas utilizan por defecto esta representación.

El objeto `bytes`

En Python los *bytes* se representan con el objeto de tipo `bytes`. Un objeto de tipo `bytes` es una secuencia **inmutable**, tal como los `str`. Para declarar que un objeto es un *byte* simplemente se pone al comienzo del objeto una `b`. Por ejemplo:

```
# Lo que está entre las comillas es un objeto de tipo bytes
# La notación \x63 indica el valor hexadecimal 63
# Este ejemplo almacena los caracteres c, l, i, c, h, é
caracteres = b"\x63\x6c\x69\x63\x68\xe9"
print(caracteres)
print(type(caracteres))
```

```
b'clich\xe9'
<class 'bytes'>
```

Cada secuencia `\x63` representa **1 Byte**. El Byte está descrito como un valor hexadecimal. En este caso el valor hexadecimal 63.

El símbolo escape `\x` indica que los siguientes dos caracteres después de la `x` corresponden a un *byte* usando dígitos hexadecimales. Los *bytes* que coinciden con los *bytes* de ASCII son

reconocidos inmediatamente, así cuando los tratamos de imprimir aparecen correctamente (`clich`), el resto (é) se imprime como hexadecimal. La `b` en la impresión nos recuerda que lo que está a la derecha es un objeto de tipo `bytes`, no un `str`.

Los *bytes* a secas pueden representar cualquier entidad, desde caracteres codificados de un *string* a pixeles de una imagen. Para poder interpretar correctamente los *bytes*, necesitamos conocer la forma en que fueron codificados. Por ejemplo, un patrón binario de 8 *bits* (1 *byte*) puede corresponder a un carácter en particular si lo decodificamos usando la codificación llamada `latin1`, pero puede corresponder a un carácter completamente distinto si lo decodificamos como un carácter de tipo `utf-16`.

```
caracteres = b"\x63\x6c\x69\x63\x68\xe9"  # Secuencia de 6 bytes
print(caracteres)

# Con bytes.decode() interpretamos los bytes utilizando la
codificación latin-1 para obtener un string
print(caracteres.decode("latin-1"))
```

```
b'clich\xe9'
cliché
```

```
# 0x61 y 0x62 son la representación en hexadecimal de los caracteres
'a' y 'b', respectivamente
caracteres = b"\x61\x62"
print(caracteres.decode("ascii"))
```

```
ab
```

```
# 97 y 98 corresponden al código ASCII (en decimal) de a y b,
respectivamente
caracteres = bytes((97, 98))
print(caracteres)
```

```
b'ab'
```

```
# Esto generará un error ya que sólo se pueden usar literales ASCII
para la creación de bytes
caracteres = b"áb"
```

```
  Cell In[9], line 2
    caracteres = b"áb"
                 ^
SyntaxError: bytes can only contain ASCII literal characters
```

El método `decode` retorna un *string* normal (Unicode). Si, por ejemplo, hubiésemos usado otro alfabeto, habríamos obtenido otro *string*.

```
caracteres = b'\x63\x6c\x69\x63\x68\xe9'
print(caracteres.decode("latin-1"))
print(caracteres.decode("iso8859-5"))
```

```
print(caracteres.decode("CP437"))
print(caracteres.decode("utf-16"))
```

```
cliché
clichщ
clich0
汉捩
```

Para codificar un *string* en un alfabeto específico, simplemente usamos el método `encode` de la clase `str`. Es necesario ingresar como argumento el conjunto de caracteres o alfabeto con que se quiere codificar.

```
caracteres = "estación"
print(caracteres.encode("UTF-8"))  # 8-bit Unicode Transformation
Format
print(caracteres.encode("latin-1"))
print(caracteres.encode("CP437"))

# No se puede codificar en ASCII el caracter "ó" ya que no existe
dentro
# de los 128 caracteres de ASCII
print(caracteres.encode("ascii"))
```

```
b'estaci\xc3\xb3n'
b'estaci\xf3n'
b'estaci\xa2n'

---------------------------------------------------------------------
-----
UnicodeEncodeError                          Traceback (most recent call
last)
Cell In[11], line 8
      4 print(caracteres.encode("CP437"))
      6 # No se puede codificar en ASCII el caracter "ó" ya que no
existe dentro
      7 # de los 128 caracteres de ASCII
----> 8 print(caracteres.encode("ascii"))

UnicodeEncodeError: 'ascii' codec can't encode character '\xf3' in
position 6: ordinal not in range(128)
```

El método `encode` nos ofrece opciones de cómo manejar el caso en que el *string* que se quiere codificar no puede ser codificado con el alfabeto requerido. Estas opciones se ingresan a través del argumento opcional `errors`, donde los valores posibles son: `strict` (el valor por defecto), `replace`, `ignore` o `xmlcharrefreplace`.

```
print(caracteres.encode("ascii", errors='replace'))  # en ascii se
reemplaza el caracter desconocido con "?"
print(caracteres.encode("ascii", errors='ignore'))
```

```
print(caracteres.encode("ascii", errors='xmlcharrefreplace'))  # se crea una entidad xml que representa el caracter Unicode
```

```
b'estaci?n'
b'estacin'
b'estaci&#243;n'
```

En general, si queremos codificar un *string* y no sabemos con qué alfabeto deberíamos codificar, lo mejor es usar UTF-8, ya que los primeros 128 caracteres de UTF-8 son los mismos que en ASCII (es *backwards* compatible con ASCII).

Recordar siempre que los objetos tipo *byte* son **inmutables**.

El objeto `bytearray`

Los *bytearrays* son **arreglos (listas) de `bytes`** que, a diferencia de los `bytes` **son mutables**. Los `bytearrays` se comportan como las listas: podemos indexar con la notación de *slicing*, y también podemos ir agregando `bytes` con el método `extend`. Para construir un `bytearray` podemos ingresar un objeto `bytes` inicial.

```
ba = bytearray(b"holamundo")
print(ba)

# Podemos ocupar la notación de slicing
print(ba[3:7])
ba[4:6] = b"\x15\xa3"
print(ba)

# Podemos agregar bytes con el método extend
ba.extend(b"programa")
print(ba)
```

```
bytearray(b'holamundo')
bytearray(b'amun')
bytearray(b'hola\x15\xa3ndo')
bytearray(b'hola\x15\xa3ndoprograma')
```

```
# Aquí se imprime un byte, representado por un entero, el ascii que corresponde a la letra "h"
print(ba[0])
# La función bin genera un string con la representación binaria del byte
print(bin(ba[0]))
# El método zfill(r) rellena con 0's al inicio hasta completar r caracteres
print(bin(ba[0])[2:].zfill(8))
```

```
104
0b1101000
01101000
```

La última línea permite imprimir directamente los valores de los *bits* correspondientes al primer *byte* (representado en el literal `h` o el entero 104). El `[2:]` es para partir desde la tercera posición, ya que las primeras dos posiciones contienen los caracteres `0b`, que simplemente indica que el formato es en binario (línea anterior). Al agregar `.zfill(8)` indicamos que se usarán 8 *bits* para representar el *byte*, lo cual tiene sentido cuando hay ceros por el lado izquierdo y el *default* no los muestra (línea anterior tiene sólo 7 bits después del `0b`).

Un caracter de un *byte* puede ser convertido a un entero usando la función `ord`.

```
# Recordemos que imprimir un elemento de un bytearray nos muestra el
int que le representa
print(bytearray(b"a")[0])
# La función ord hace esta conversión directamente a partir de un byte
print(ord(b"a"))
```

```
97
97
```

```
# Aqui ocupamos ambas formas (int directamente y ord) para hacer
cambios en un bytearray
b = bytearray(b'abcdef')
b[3] = ord(b'g')         # El caracter g tiene como código ascii el
103
b[4] = 68                # El caracter D tiene como código ascii el
68, esto sería lo mismo que ingresar b[4] = ord(b'D')
print(b)
```

```
bytearray(b'abcgDf')
```

Como mencionamos antes, los `bytearrays` se comportan como las listas. Por lo tanto, podemos hacer todas las operaciones que hemos aprendido en listas:

```
mi_bytearray = bytearray()

# El método append solo funciona en bytearray con un int
mi_bytearray.append(2)
print(mi_bytearray)
```

```
bytearray(b'\x02')
```

```
mi_bytearray = bytearray()

# El método extend funciona con byte o bytearray
mi_bytearray.extend(b'\xff') #extendemos con un byte
print(mi_bytearray)

mi_bytearray.extend(bytearray(b'\xff\x12')) #extendemos con otro
bytearray
print(mi_bytearray)
```

```
bytearray(b'\xff')
bytearray(b'\xff\xff\x12')

mi_bytearray = bytearray()

mi_bytearray.append(44) # Append de un `int`
print(mi_bytearray[0]) # Veremos el número (no el byte)

44

mi_bytearray = bytearray()

mi_bytearray.append(1) # Append de un `int`
mi_bytearray.append(2) # Append de un `int`
mi_bytearray.append(3) # Append de un `int`
print(mi_bytearray)
print(mi_bytearray[0: 2]) # Veremos el bytearray en esa sección

bytearray(b'\x01\x02\x03')
bytearray(b'\x01\x02')

mi_bytearray = bytearray(b'\x01\x02\x03')

# Veremos el `int` más grande entre cada byte del bytearray
print(max(mi_bytearray))

3

mi_bytearray = bytearray(b'\x01\x02\x03')

for x in mi_bytearray:
    print(x) # X será cada `int` del bytearray

1
2
3
```

Chunks

Un concepto importante en el manejo de *bytes* es lo que se entiende por ***chunk***: un grupo de *bytes*. Cuando trabajamos con una cantidad importante de *bytes*, no es conveniente leerlos todos a la vez, ya que pueden ser muchos y no caber en nuestra memoria; ni tampoco uno por uno, pues requiere más operaciones de lectura de datos y puede ser muy lento. Sin embargo, podemos utilizar el punto medio: leer los *bytes* en *chunks* (grupos) de cierta cantidad de *bytes*. Por ejemplo, el siguiente *bytearray* lo leeremos en *chunks* de tamaño 4 (`TAMANO_CHUNK`):

```
muchos_bytes = bytearray(b"Una gran cantidad de bytes que quiero leer
de a poquito")
TAMANO_CHUNK = 4

for i in range(0, len(muchos_bytes), TAMANO_CHUNK):
```

```
    # Aqui obtenemos nuestro chunk
    chunk = bytearray(muchos_bytes[i:i+TAMANO_CHUNK])
    print(chunk)
```

```
bytearray(b'Una ')
bytearray(b'gran')
bytearray(b' can')
bytearray(b'tida')
bytearray(b'd de')
bytearray(b' byt')
bytearray(b'es q')
bytearray(b'ue q')
bytearray(b'uier')
bytearray(b'o le')
bytearray(b'er d')
bytearray(b'e a ')
bytearray(b'poqu')
bytearray(b'ito')
```

También podemos leer los archivos por chunks, usando el modo de lectura de bytes `rb` al abrir el archivo:

```
from os.path import join

TAMANO_CHUNK = 4
with open(join('data', 'ejemplo_chunk.bin'), 'rb') as file:
    while chunk := file.read(TAMANO_CHUNK):
        print(chunk)
```

```
b'IIC2'
b'233 '
b'Prog'
b'rama'
b'ci\xc3\xb3'
b'n Av'
b'anza'
b'da'
```

Al usar el `file.read()` pasándole un número como parámetro, se puede leer en bloques de esa cantidad de bytes.

> (Inicio paréntesis
>
> Aquí hemos usado el llamado 'walrus operator' `:=` que almacena el valor de lo que hay en la derecha en lo que hay a la izquierda, de esta forma, guardamos en `chunk` lo leido, y si lo leído es vacío (por ejemplo si se leyó todo el archivo) entonces finaliza el `while` al recibir algo vacío. De esta forma lo que se hace es guardar en la variable `chunk` el valor de `file.read(TAMANO_CHUNK)`, y también se usa para revisar el valor en el `while`. Por lo anterior el código se ha vuelto menos engorroso al no hacer

las dos operaciones por separado, pero hay que tener cuidado de no usarlo demasiado ya que puede causar un código menos legible.

Fin paréntesis)

Transformar números

El tipo de dato `int` como objeto tiene métodos adicionales que nos permiten entender la representación de este en *bytes*. Estos métodos pueden ser útiles cuando queremos trabajar con data binaria, o cuando usemos protocolos que requieran codificar y decodificar enteros (*integers* o `int`) a *bytes*. En esta conversión, será importante establecer cómo se ordenarán los bytes en memoria al representar datos de múltiples bytes como lo sería un *integer*.

Dado lo anterior, es necesario entender el concepto de `byteorder`. Como un 1 *byte* posee 8 *bits*, como máximo puede representar un número entre 0 y 255 (`0x00` es `0` y `0xFF` es `255`). Por lo tanto, ¿cómo se transforma un número más grande a *bytes*?

Aquí surge el concepto de *big endian* y *little endian*.

- En *big endian* el *byte* más significativo (de mayor peso) quedará al inicio del *byte array*. Por ejemplo, el número 256 en *big endian* es `\x01 \x00`.
- En *little endian* es lo opuesto, el *byte* más significativo (de mayor peso) quedará al final del *byte array*. Por ejemplo, el número 256 en *little endian* es `\x00 \x01`.

Veamos algunos ejemplos más:

Número decimal	*Byte array* en *big endian*	*Byte array* en *little endian*
269	`\x01\x0d`	`\x0d\x01`
68866	`\x01\x0d\x02`	`\x02\x0d\x01`
1157698	`\x11\xAA\x42`	`\x42\xAA\x11`

Ahora, veamos cómo podemos transformar números en *byte* mediante código:

- `int.to_bytes(length = 1, byteorder = 'big')`: retorna el *integer* como un arreglo de *bytes*. Debe recibir al menos dos argumentos: el largo del *byte array* y el orden. Si `byteorder` es 'big', el *byte* más significativo (de mayor peso) quedará al inicio del *byte array*. Si `byteorder` es 'little', el *byte* más significativo (de mayor peso) quedará al final del *byte array*.

  En caso que `length` sea más grande que la cantidad necesaria de bytes para representar el número, esta función rellenará con los 0 necesarios para que el valor no cambie, pero pueda alcanzar el largo indicado. Esto último es análogo a cuando escribimos `001235`, donde agregamos 2 ceros que no influyen en el valor.

```
(65).to_bytes(1, 'little')
```

```
b'A'
```

```
(256).to_bytes(2, 'big')
```

```
b'\x01\x00'
```

```
(256).to_bytes(5, 'big')
```

```
b'\x00\x00\x00\x01\x00'
```

```
(256).to_bytes(2, 'little')
```

```
b'\x00\x01'
```

```
(256).to_bytes(5, 'little')
```

```
b'\x00\x01\x00\x00\x00'
```

```
i = 1024
i.to_bytes(4, byteorder='big') # prueba cambiando el segundo argumento a 'little'
```

```
b'\x00\x00\x04\x00'
```

- `int.from_bytes(byte, byteorder = 'big')`: retorna el *integer* representado por el arreglo de *bytes*. Debe recibir al menos dos argumentos: el *byte* (cualquier objeto del tipo byte o un iterable que produzca *bytes*) y el orden. Si `byteorder` es 'big', el *byte* más significativo (de mayor peso) será el del inicio del *byte array*. Si `byteorder` es 'little', el *byte* más significativo (de mayor peso) será el del final del *byte array*.

```
int.from_bytes(b'\x00\x00\x04\x00', byteorder='big')
```

```
1024
```

```
int.from_bytes(b'\x00\x00\x04\x00', byteorder='little')
```

```
262144
```

```
int.from_bytes(b'A', byteorder='little')
```

```
65
```

```
int.from_bytes(b'A', byteorder='big')
```

```
65
```

Podrás notar en el ejemplo anterior que cuando el largo del *byte array* es 1, no influye el `byteorder`. No obstante, cuando el *byte array* tiene un largo mayor a uno, pasar de `big` a `little` puede cambiar radicalmente el valor del *integer* a retornar.

Print de `bytes`

Un mensaje muy importante a transmitir cuando trabajamos con *bytes* es: **No confíes en los `print` de bytes**.

Cuando transformamos un número en *bytes* e imprimos en consola ese valor, el computador puede reemplazar **visualmente** el *byte* a mostrar por otro carácter. Esto no implica que está mal la operación, solo es una forma que el computador tiene para representar cierto valor de byte cuando hace `print`.

Veamos un ejemplo:

El byte `\x09` es `9`. Usemos el método `from_bytes` podemos confirmar eso:

```
int.from_bytes(b'\x09', byteorder='big')
```

```
9
```

Pero ahora transformemos ese 9 en un *byte*

```
(9).to_bytes(1,'big') # Esto debería ser \x09
```

```
b'\t'
```

Que raro... sale que 9 es '\t'. Esto no está mal, es solo que visualmente `'\t'` equivale a 9. Incluso, si usamos el método `from_bytes`, podemos confirmar que es 9.

```
int.from_bytes(b'\t', byteorder='big')
```

```
9
```

Esto mismo ocurre con el 13:

```
int.from_bytes(b'\x0d', byteorder='big')
```

```
13
```

```
(13).to_bytes(1,'big') # Esto debería ser \x0d
```

```
b'\r'
```

```
int.from_bytes(b'\r', byteorder='big')
```

```
13
```

Por lo tanto, cuando trabajamos con *bytes*, recomendamos no depender de cómo visualmente se ve el *byte*, sino transformar ese *byte* en el *integer* respectivo y con eso verificar que sea el número esperado.